

Which local does the model return?

Per-name pathways for Python variables in gemma-2-2b

Anna Liang ■ PyCon TW 2026 ■ mechanistic interpretability with circuit-tracer + GemmaScope transcoders

model google/gemma-2-2b, transcoders mwhanna/gemma-scope-transcoders, RTX 3090, bfloat16 (never float16 — gemma-2’s activations overflow it). One model only: nothing here is claimed to generalise across models.

1 The question

Cut a Python function off right after return and the model must choose *which local variable to return*:

```
def calculate_total(prices):      # P1
    result = 0
    for price in prices:
        result = result + price
    return
```

gemma-2-2b answers ’ result ’ at **95.7%**. Is that a representation of *binding* — or just of the name being on the page?

2 The controls: binding, not occurrence

P2/P3 dissociate the two. They are **deliberately invalid Python** (a name is read before assignment) — and the valid-Python control below reproduces everything, so nothing rests on broken code.

```
def calculate_total(prices): # P2: 68.8 / 25.2
    ans = 0                  # bound, never accumulated
    for price in prices:
        result = result + price
    return
def calculate_total(prices): # P3: 89.1 / 6.5
    ans = 0
    for price in prices:
        ans = result + price # result is only ever READ
    return
def calculate_total(prices): # two-valid: 82.4 / 11.2
    ans = 0                  # VALID PYTHON
    result = 0
    for price in prices:
        result = result + price
    return
```

P3 settles it: a name that is present but never bound loses, 89.1 to 6.5.

Not about the name result — 18 names × 4 patterns:

pattern	valid?	predicts	top-1	mean margin
consistent (A bound twice)	yes	’ A ’	18/18	+94.8 pp
two-valid (both bound, A grows)	yes	’ A ’	18/18	+59.6 pp
never-bound (A only read)	control	’ D ’	18/18	+89.0 pp
split (D line 2, A in loop)	control	’ A ’	17/18	+26.5 pp

The single miss (score, split) is an *exact* 44.92/44.92 tie — reported as a miss, not tidied away.

3 Mechanism: one pathway per name, and they are orthogonal

Direct effect of every node at each name’s positions, summed, measured **against each logit separately** (five two-candidate graphs):

graph	winner	→own	→other	loser	→own	→other
split-binding	result	+26.96	+1.39	ans	+20.27	+0.05
result-never-bound	ans	+26.96	+0.70	result	+17.03	+0.92
two-valid-locals	result	+37.30	+2.00	ans	+21.30	−0.21
split-total	total	+38.88	−0.64	ans	+19.00	+0.06
total-never-bound	ans	+28.04	+0.66	total	+20.84	−0.28

Against own-drive of 17–39, **no cross term exceeds 2.0**. No gate, no mutual suppression: binding sets how much drive a name accumulates, then the logit layer does winner-take-all.

18_4769 is the *result* pathway; 18_4732 is *ans*’s. 18_4769 does *not* respond to *ans* = 0 — a perfectly good binding — so the features are tied to the *name*, not to the act of binding. 24_12518 at return adds +1.68 to ’ result ’ *and* +1.78 to ’ ans ’ in P2: name-agnostic, belongs to neither.

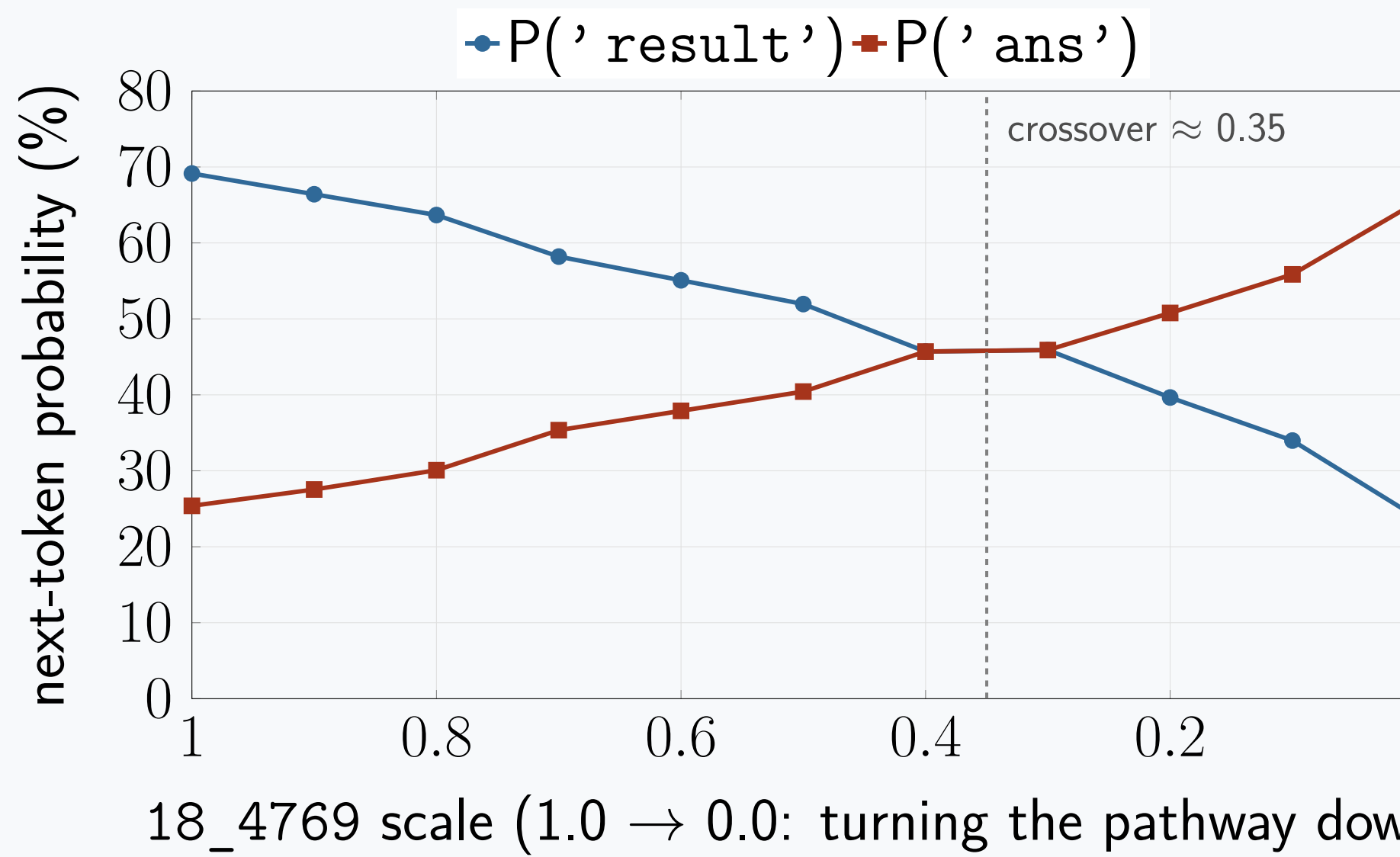
Each well-covered pathway has its own suppressor (layer 24, at return):

pathway	suppressor	ratio to own drive
result	24_15424	0.285 / 0.263 / 0.240 / 0.292
total	24_12843	0.285 / 0.288
ans	<i>not captured</i>	largest negatives 0.06–0.10

A *different* feature opposes a *different* name: per-pathway gain control, not one global calibration. *ans*’s drive sits in embeddings (+7.4) rather than its feature (+1.7) — the transcoders cover that pathway poorly, so say “not captured”, never “does not exist”. The 0.24–0.29 ratio is an observation, not a law.

4 Causal proof: turn the volume down and the answer flips

Scaling 18_4769 (the *result* pathway) down in P2, every step inside the feature’s natural activation range:



Monotonic, crossing at ≈0.35. (The 0.4/0.3 steps read as exact ties because of bf16 quantisation, not a flat curve.) Ablations agree:

condition	P(’ result ’)	P(’ ans ’)	argmax
P2 baseline	69.14	25.39	’ result ’
P2 ablate 18_4769	24.02	65.23	’ ans ’ — flipped
P2 ablate 18_4732	73.44	21.00	’ result ’
P1 ablate 18_4769	95.31→69.92	0.00	’ result ’
P3 ablate 18_4769	6.45→0.93	94.53	’ ans ’
P3 ablate 18_4732	8.06	89.06→86.72	’ ans ’

The last row converges: removing *ans*’s feature barely moves ’ ans ’, exactly as attribution predicted (its drive sits in embeddings). **Two independent methods, same answer.** The baseline forward pass reproduces the behavioural probe (95.31 / 69.14 / 6.45) first.

5 What we got wrong, twice — and how to not do that

The central claim was wrong twice, and one supporting number a third time. The correction *is* the story.

Retracted, refuted by our own data:

- “18_4769 tracks the bound identifier” — it ignores *ans* = 0, a perfectly good binding.
- “Binding gates whether the signal is routed” — the first correction, also wrong.
- “18_4769 fires but is not routed” — its +0.028 was measured against the *wrong logit*. Against ’ result ’ it is **+8.500**.
- “18_4732: identical activation, opposite routing” — two numbers against two *different* logits. Same logit: +1.56 vs +1.31.
- “Winner’s drive is conserved” — two points agreed to three digits; five graphs spread **11.9**. Two points are not a law.

The traps, stated so you can reuse them:

- **A direct effect is meaningless until you name the logit it is against.** This one mistake produced retractions 2–4. Every number on this poster names its target.
- **The pinned subset can invert the answer.** In P3 the pinned nodes give *result* +12.8 vs *ans* +11.9 — opposite to the 89/6 outcome. Truth appears only when every node is summed (+26.96 vs +17.03; *ans* is spread over 500 nodes to 102). Any subgraph figure needs the aggregate totals beside it.
- **Amplifying misleads.** 18_4732 up 8× *decreases* P(’ ans ’) to 0.1% — the residual stream leaves distribution. Scale down, never up.
- **P1’s aggregate is not comparable** — its graph has a single logit target (−desired-logit-prob 0.95); keep it out of the table.
- The conservation downgrade was **pre-registered** before the new data existed — the only reason it was retired rather than defended.

6 Where it breaks: binding across time, scope, and width

The model has a clean representation of *what a name is bound to*. It does not have one for *when the binding happens, or in which scope* — exactly the gap an AI code debugger has to close. All ground truth below is obtained by **executing the snippet** (under the exact interpreter and environment each case names); the probes refuse to start if Python disagrees.

Generator-expression late binding (PEP 289). Real teaching-video snippets, cut at single-token divergence points:

cut	correct	model	
no-reassignment control	,	98.4%	ok
which list is iterated (genexp)	1	0 at 55.1%	MISS
which list is iterated (desugared)	1	40.4 / 40.4	exact tie
filter’s list (genexp)	]	53.1% (+6.3 pp)	ok
filter’s list (desugared)	]	73.1% (+46.1 pp)	ok
does (3, ...) appear	,	0.00%	MISS

PEP 289 *defines* the *genexp* to be its desugaring — one program, written twice — yet the model is **worse on the sugared form on both cuts**. The only cut that cleanly separates eager from lazy capture gets **zero probability**. (This supports surface-form reading; an exact tie has no answer to contradict, so it does not prove self-contradiction.)

Ten classic binding traps — pairs, or the score lies. Near-identical surface forms, opposite answers. Per-case 5/10; **pairs 0/4**:

pair	what the model does
lambda: i vs lambda i=i: i	says [0, 1, 2] to both
acc=[] vs acc=None	says [1, 2] to both
loop-var leak: for vs comprehension	exactly inverted
class-body comprehension, illegal vs legal	legal ok; illegal at noise

The leak pair is the sharpest: the *for* variable *does* leak, model says *Traceback* (35.6%); the comprehension variable does *not*, model says 2 (33.6%). Both rules, swapped, with matching confidence.

Four chapters of “the expression is not enough” — 1/13 pairs. floats/inf/nan; set hash order vs dict insertion order; bound-method identity (CPython behaviour, not a language guarantee); the same *locals()* snippet under 3.11/3.12/3.13 (PEP 709).

- ch.1 fits **literal identity, not arithmetic**: same expression twice → True, different literals → False — so 1e500 == 1e600 gets False at 67.6% (both are inf).
- ch.4: it answers ’k’ for every version — right for 3.12/3.13, and 0.00% on the ’.0’ that 3.11 actually prints. One answer per snippet, no model of interpreter versions.
- Methodology: on bare »> prompts the top token was »> itself (37–45%) — the model skipped answering. Fixed with counterbalanced two-shot priming; a verdict that flips with primer order scores *prime-dep*, not a win. 0.1 + 0.2 == 0.3 is one.

Explore the graphs live

Interactive attribution graphs on Neuronpedia (pins and group labels encoded in the URLs; the **two-valid-locals** graph — valid, runnable Python — is the one to start with; its cross terms are +0.026 and +0.063, and it contains the project’s two largest nodes, 18_4769_16 at +10.56 driving ’ result ’ and 24_15424_36 at −10.88 opposing it, very nearly cancelling):

neuronpedia.org/gemma-2-2b/graph?slug=py-two-valid-locals py-return-variable py-split-binding py-result-never-bound

In every graph, **all logit nodes are pinned**, including ’ (’ at 1.3% — the names compete inside a real next-token distribution, not a two-way choice we constructed.

Write-up and self-hosted viewer: **ebenezer61.github.io/interpretability/**

Every number on this poster lives in a committed file under experiments/outputs/; probes execute their own ground truth before scoring. Poster built 2026-08-27.